Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Отчет по лабораторной работе №3
по курсу
”Математические основы верификации ПО”

Тема Моделирование сетевого протокола

Студент Ву Минь Куанг

Группа ИУ7и-42М

Оценка (баллы) _____

Преподаватели Кузнецова О.В.

Москва — 2026 г.

Оглавление

Введение **3**

1 Выполнение **4**

1.1 Описание протокола TCP 4

1.2 Описание модели и результаты 5

ЗАКЛЮЧЕНИЕ **10**

ВВЕДЕНИЕ

Цель работы – описать упрощенную модель протокола TCP.

Для достижения поставленной цели потребуется:

- описать протокол и принятые допущения;
- привести описываемые UML-sequence;
- привести модель протокола;
- привести логи SPIN, демонстрирующие отправку/получение данных: пакетов, HTML-документов и прочего.

1. Выполнение

1.1 Описание протокола TCP

TCP (Transmission Control Protocol) - это протокол транспортного уровня в сетях TCP/IP. Он обеспечивает надежную, упорядоченную и точечную передачу данных между устройствами в сети. TCP гарантирует доставку данных без потерь, дублирования или искажения в правильном порядке. Для этого используются подтверждения и механизм повторной отправки данных в случае возникновения ошибок. Кроме того, данный протокол контролирует скорость передачи данных между отправителем и получателем для предотвращения переполнения буфера или перегрузки сети – это достигается за счет использование механизмов окна передачи и алгоритмов обратной связи. TCP устанавливает и разрывает виртуальные соединения между устройствами для передачи данных, что включает в себя процедуры установки и разрыва соединения с использованием трехстороннего рукопожатия. Каждое приложение на устройстве использует порт для связи в сети, а TCP использует номера портов для адресации различных служб и приложений. При передаче данные сегментируются, что позволяет оптимизировать использование сетевых ресурсов и обеспечивает более надежную передачу данных.

UML-sequence диаграмма протокола представлена на рисунке 1.1. В качестве допущения в реализуемой модели определим, что:

- модель предусматривает отправку в качестве данных только целочисленных значений, причем в самой модели будет описана отправка всего двух элементов;
- модель не предусматривает проверки целостности данных и корректности значений, для которых на схеме задействованы параметры *i, j, a, b, c, n, m*;
- в модели не будет учитываться поле пакетов *SEQ*;
- в качестве моделирования отправки одновременно двух флагов в одном пакете, например *SYN* и *ACK*, будут использоваться такие типы как *SYN_ACK*.

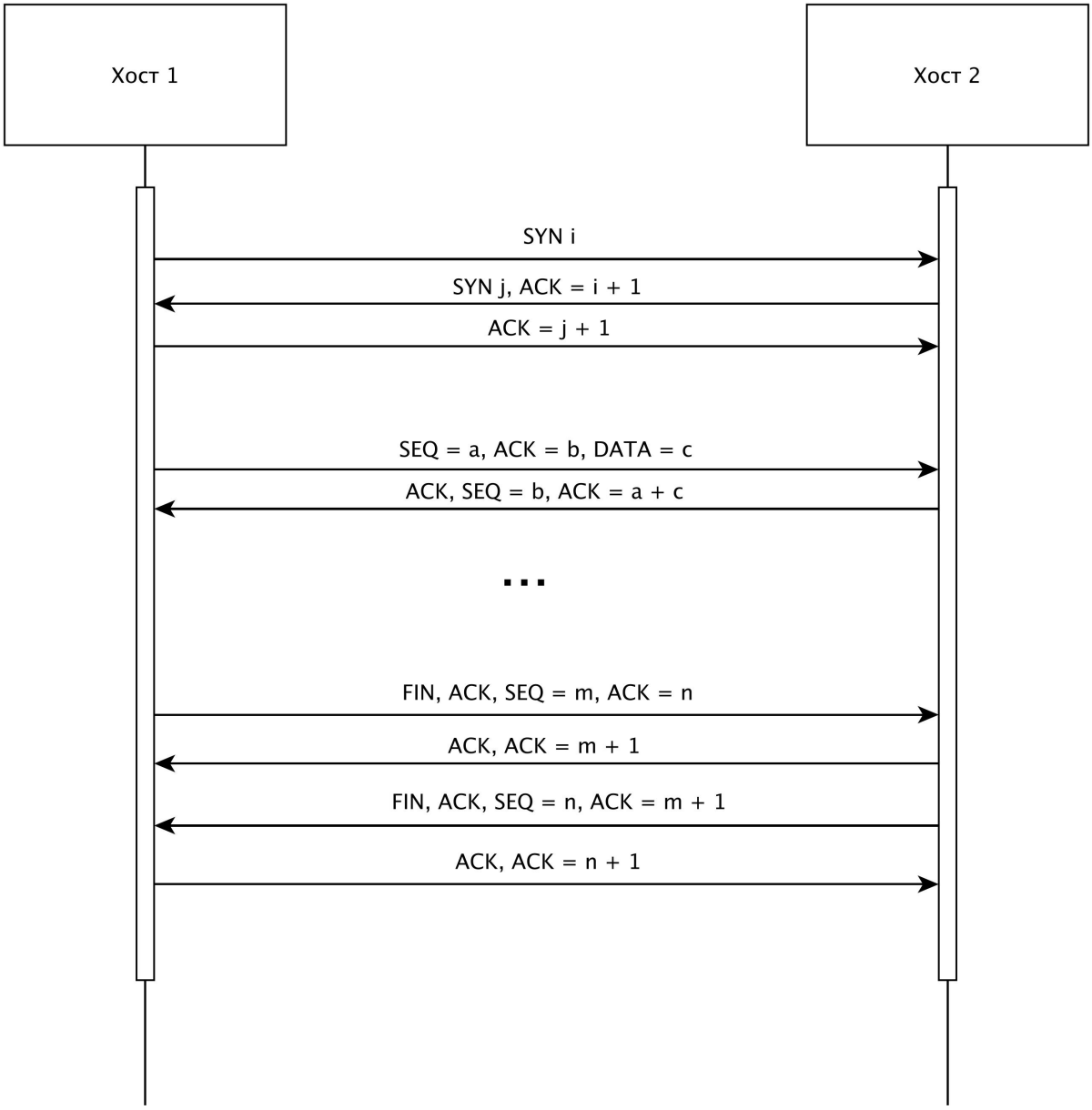

Рис. 1.1: UML-sequence диаграмма взаимодействия двух хостов с использованием протокола TCP.

1.2 Описание модели и результаты

Описание модели представлено в листинге 1.

Листинг 1: Описание модели взаимодействия процессов.

```
#define MAX_BUF 2
#define DATA_MESSAGES_COUNT 2

```

```
mtype {DATA, ACK, SYN, SYN_ACK, FIN_ACK};

chan client_to_server = [MAX_BUF] of {int, mtype};
chan server_to_client = [MAX_BUF] of {int, mtype};

proctype Client() {
    int data = 1337;
    int seq_num = 0;
    mtype msg_type;

    printf("The client sends a SYN.\n");
    client_to_server ! data, SYN;

    server_to_client ? _, msg_type;
    if
    :: (msg_type == SYN_ACK) ->
        printf("The client receives a SYN-ACK from the server and sends an ACK in response.\n");
        client_to_server ! 0, ACK;
    fi

    do
        :: seq_num < DATA_MESSAGES_COUNT ->
            printf("The client sends data %d\n", data);
            seq_num++;
            msg_type = DATA;
            client_to_server ! data, DATA;
            data++;
            server_to_client ? _, msg_type;
            if
            :: (msg_type == ACK) ->
                printf("The client receives an ACK from the server\n");
            fi

        :: else ->
            break;
    od

    server_to_client ? _, msg_type;
    if
```

```
        :: (msg_type == ACK) ->
            printf("The client receives an ACK from the server\n");
        fi

        printf("The client sends a FIN_ACK to the server\n")
        client_to_server ! 0, FIN_ACK;

        server_to_client ? _, msg_type;
        if
        :: (msg_type == ACK) ->
            printf("The client received an ACK from the server\n")
        fi

        server_to_client ? _, msg_type;
        if
        :: (msg_type == FIN_ACK) ->
            printf("The client received a FIN_ACK from the server\n")
        fi

        printf("The client sends the final ACK to the server and terminates its operation\n")
        client_to_server ! 0, ACK;
    }

    proctype Server() {
        int received_data;
        mtype msg_type;

        client_to_server ? _, msg_type;
        if
        :: (msg_type == SYN) ->
            printf("The server receives a SYN from the client and sends a SYN_ACK in response\n");
            server_to_client ! 0, SYN_ACK;
        fi

        client_to_server ? _, msg_type;
        if
```

```
        :: (msg_type == ACK) ->
            printf("The server receives an ACK from the client\n");
            server_to_client ! 0, ACK;
        fi

        printf("The server is waiting for data\n");
        client_to_server ? received_data, msg_type;

        do
            :: (msg_type == DATA) ->
                printf("The server receives data %d and sends an ACK back to the client\n", received_data);
                server_to_client ! 0, ACK;
                client_to_server ? received_data, msg_type;

            :: (msg_type == FIN_ACK) ->
                printf("The server receives a FIN_ACK from the client and sends an ACK and a FIN_ACK to the client\n")
                server_to_client ! 0, ACK;
                server_to_client ! 0, FIN_ACK;
                break;
        od

        client_to_server ? 0, msg_type;

        if
        :: (msg_type == ACK) ->
            printf("The server received an ACK from the client and is shutting down\n")
        fi
    }

    init {
        atomic {
            run Client();
            run Server();
        }
    }
```

На рисунке 1.2 представлены логи SPIN, демонстрирующие работу модели.

```
E:\4_semester_magis\SoftwareVerification\lab3>spin lab.pml
          The client sends a SYN.
              The server receives a SYN from the client and sends a SYN_ACK in response
          The client receives a SYN-ACK from the server and sends an ACK in response.
              The server receives an ACK from the client
          The client sends data 1337
              The server is waiting for data
              The server receives data 1337 and sends an ACK back to the client
          The client receives an ACK from the server
          The client sends data 1338
              The server receives data 1338 and sends an ACK back to the client
          The client receives an ACK from the server
          The client receives an ACK from the server
          The client sends a FIN_ACK to the server
              The server receives a FIN_ACK from the client and sends an ACK and a FIN_ACK to the client
          The client received an ACK from the server
          The client received a FIN_ACK from the server
          The client sends the final ACK to the server and terminates its operation
              The server received an ACK from the client and is shutting down
3 processes created
```

Рис. 1.2: Логи SPIN, демонстрирующие работу модели.

ЗАКЛЮЧЕНИЕ

В результате выполнения работы была описана упрощенная модель протокола TCP на ЯП Promela.

Были решены следующие задачи:

- описан протокол TCP и принятые допущения;
- приведено описание EML-sequence;
- приведена модель протокола;
- приведены логи SPIN, демонстрирующие отправку/получение данных.